# DSCI 551 Course Project Final Report

---

**Project Title**: Exploring PostgreSQL Indexing and Query Execution through a Flight Search Application

**Name:** Rylan Nathan Lewis

**USC ID:** 8358130873

---

## 1. Introduction & Motivation:

Modern data-intensive applications depend heavily on the performance characteristics of the underlying database system. This is highly evident in real-time search platforms, where users expect sub-second responses to complex multi-condition queries across millions of records. In this project, the degree of influence of the characteristics of the database system performance on modern applications dealing with large amounts of data will be studied.

The proposed application will represent a simplified airline search system using actual historical flight data provided by the Bureau of Transportation Statistics (BTS). Searches are performed to retrieve information about a specific flight between airports, filter dates and departures by time, sort by price or duration of the trip, and choose the best one among possible options. Each of the aforementioned capabilities represents some kind of database query and requires the presence of appropriate indexes to function efficiently.

PostgreSQL database was chosen owing to its clear understanding of query execution algorithms. The use of such features as EXPLAIN and EXPLAIN ANALYZE shows the query planner's decisions, helping us understand which scanning method (sequential, index, or index-only) was applied, how many rows were analyzed, and how long the process took. This visibility makes PostgreSQL uniquely well-suited for mapping the relationship between application queries and underlying database behavior.

---

## 2. Database System Overview:

PostgreSQL is an open-source, fully ACID-compliant relational database management system. This software provides a wide range of datatypes, index facilities, text searching, JSON, and procedural languages such as PL/pgSQL. Since it was built to be accurate and extensible, PostgreSQL has become a popular choice for use in both academic studies and practical applications involving complicated analytics and transactions.

Key characteristics of PostgreSQL, of which some are relevant to this project, include:

1. B-tree Indexes: Advanced B-tree indexing with support for composite, partial, and covering indexes.
2. Query Planner: A cost-based query planner that selects execution strategies based on table statistics, available indexes, and estimated row counts.

3. MVCC: Multi-Version Concurrency Control (MVCC), which allows readers and writers to operate concurrently without blocking each other.
4. Query Introspection: The EXPLAIN and EXPLAIN ANALYZE commands, which expose internal execution plans, buffer usage, and actual timing data for any query.
5. Specialized Indexes: Support for partial indexes (with WHERE clauses) and covering indexes (with INCLUDE clauses) that optimize specific query patterns.

In terms of intended use cases, PostgreSQL excels in OLTP (Online Transaction Processing) systems requiring strict data consistency, complex multi-table queries, and concurrent access. The project uses PostgreSQL installed locally. A database server named “DSCI551” is created containing the database “flightmgmtsys”. Inside this DB, the primary table is “flights”, containing approximately 5 million rows sourced from BTS monthly flight records, spanning 2023-2025.

---

## 3. Internal architecture:

1. **Storage Model:** PostgreSQL stores table data in heap files, which are collections of fixed-size pages (typically 8 KB each). Records (tuples) are written into pages in insertion order, without any inherent physical ordering by primary key or any other column. When a query filters or sorts on a column, PostgreSQL must either scan all heap pages sequentially or use a secondary index to locate the relevant rows.
   This heap-based model contrasts with clustered storage systems like MySQL's InnoDB, where rows are physically ordered by primary key. In PostgreSQL, even when an index is used, the query engine typically follows a two-step process; first traversing the index B-tree to find row locations (TIDs) and then fetching the actual data from the corresponding heap pages. The VACUUM ANALYZE command updates the visibility map, which PostgreSQL uses to enable index-only scans that bypass heap access entirely when all required columns are available in the index.
2. **Indexing:** B-tree indexes are the default and most widely used index type in PostgreSQL. They maintain a balanced tree structure where each leaf node stores index key values along with pointers (TIDs) to the corresponding heap rows. B-tree indexes support equality filters, range queries, and ORDER BY operations efficiently. This project creates and evaluates four index types across six features:
   a. Standard composite index on (origin, dest, fl_date, price): Supports route and date filtering with price-ordered retrieval.
   b. Composite index on (origin, dest, fl_date, scheduled_elapsed_minutes): Supports duration-ordered retrieval for fastest-flight queries.
   c. Partial index on (origin, dest, fl_date, price) WHERE cancelled = false: Reduces index size by including only non-cancelled flights, improving scan efficiency for the most common query pattern.
   d. Covering index using INCLUDE (scheduled_elapsed_minutes): Stores an additional column in the index leaf nodes, enabling index-only scans that never touch the heap.
   e. Departure-time index on (origin, dest, fl_date, scheduled_dep_time): Supports time-range filtering.

f. Carrier-cancellation index on (origin, dest, carrier, cancelled): Supports aggregation queries grouping by carrier.

Composite index column ordering follows the leading-column principle of PostgreSQL wherein it can use a composite index only if the query's WHERE clause includes the leftmost column(s) of the index. For example, a query filtering only on fl_date without origin cannot use an index built on (origin, dest, fl_date, price).

3. **Query Execution:** PostgreSQL uses a cost-based query planner that evaluates multiple candidate execution plans and selects the one with the lowest estimated cost. Cost estimates are based on table statistics maintained by ANALYZE, including row counts, column cardinality, and data distribution histograms. The planner chooses among three primary scan strategies:
   a. Sequential Scan: Reads all heap pages in order. Used when the query returns a large fraction of rows or when no suitable index exists. Cost scales linearly with table size.
   b. Index Scan: Traverses the B-tree index to find matching TIDs, then fetches rows from the heap. Efficient for high-selectivity queries returning a small fraction of rows. May involve random I/O for heap access.
   c. Index-Only Scan: Uses the index exclusively when all required columns are present in the index (including via INCLUDE). Avoids heap access entirely, dramatically reducing I/O. Requires an up-to-date visibility map which is maintained by VACUUM ANALYZE.

   EXPLAIN ANALYZE was used throughout this project to observe actual execution plans, buffer hit counts, planning time, and execution time, both before and after index creation, thus providing direct experimental evidence of how indexing affects query performance.

---

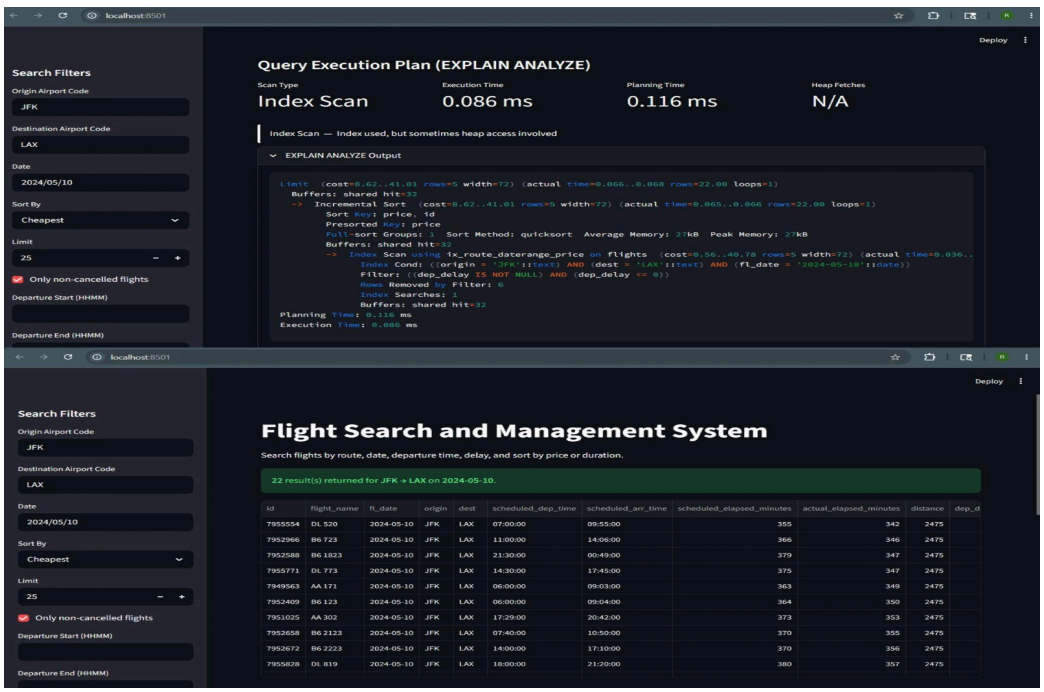
## 4. Application Design:

1. **Data Pipeline**: Raw CSV files were downloaded from the BTS Reporting Carrier On-Time Performance dataset for 2023-2025 ([https://www.transtats.bts.gov/](https://www.transtats.bts.gov/)). A Python preprocessing script (clean_csv.py) was used to standardize and load this data. The script retains 16 relevant columns, renames them to match the database schema, converts HHMM-formatted time integers to HH:MM:SS strings, builds valid fl_date values from fallback year/month/day columns where needed, and generates a synthetic price attribute using the formula,price = (distance × 0.10) + (duration × 0.50). A flight_name field combining carrier code and flight number is also derived. The cleaned output is a set of CSV files ready for direct PostgreSQL import.
2. **Database Schema and feature queries workflow**: The flights table schema is defined as follows:

   *id bigserial PRIMARY KEY, fl_date date, carrier text, flight_num int,*
   *origin text, dest text, scheduled_dep_time time, scheduled_arr_time time,*
   *scheduled_elapsed_minutes int, actual_elapsed_minutes int, distance int,*
   *dep_delay int, arr_delay int, cancelled boolean, price numeric, flight_name text*

For each feature, a before and after EXPLAIN ANALYZE is run to observe query planner behaviour in choosing the most time and cost-efficient scan after creating indexes.

3. **User Interface**: The front end has been developed using Streamlit, which connects to the PostgreSQL database using psycopg2. A side panel provides the options to filter by origin, destination, date, sort order, number of results, a checkbox for filtering out cancelled flights, a range of departure times, and a maximum delay. The application constructs a parameterized query dynamically depending on the filters active, displaying the results in an interactive dataframe. The stability and security of the system are ensured by caching connections and using parameterized queries. The live query plan window below the results section runs EXPLAIN (ANALYZE, BUFFERS) on the same query and displays the type of scan performed, elapsed time, planning time, and heap fetches as summary cards colored by the type of scan performed. The before/after EXPLAIN comparison can be reproduced by following the workflow documented in the repository README.




---

## 5. Mapping Internals to Application Behavior:

1. **Feature 1: Route + Date Search (Index Scan vs Sequential Scan):**
   - **Application behavior**: The user searches for flights between two airports on a specific date, sorted by price. The query filters on origin, dest, fl_date, and cancelled, then orders by price.

     *SELECT id, fl_date, origin, dest, scheduled_dep_time, price, scheduled_elapsed_minutes*
     *FROM flights WHERE origin='JFK' AND dest='LAX' AND fl_date='2025-01-10' AND cancelled=false ORDER BY price LIMIT 50;*

- **Mapping:** PostgreSQL uses heap storage with secondary B-tree indexes. The index created was ix_route_date_price ON flights (origin, dest, fl_date, price). Without index, the planner performs a sequential scan over all heap pages, followed by an explicit sort on price. The composite index allows the planner to use an index scan that directly retrieves rows pre-ordered by price, eliminating the sort step. Execution time was 1224.338ms.
- **Observation:** Indexing reduced no. of scanned rows and execution time dropped from 1224.338ms to 0.234 ms using Index Scan, a drastic reduction. This demonstrates that composite B-tree indexes with the sort column as the trailing key can eliminate ORDER BY overhead entirely.

2. **Feature 2: Fastest Flight Sort (Composite Index Ordering):**
   - **Application behavior:** Retrieve flights sorted by scheduled_elapsed_minutes to find the fastest travel option on a given route and date.

     *SELECT id, fl_date, origin, dest, scheduled_elapsed_minutes, price FROM flights*
     *WHERE origin='BOS' AND dest='ORD' AND fl_date='2025-02-14' AND cancelled=false*
     *ORDER BY scheduled_elapsed_minutes LIMIT 20;*
   - **Mapping:** Index *ix_route_date_duration ON flights (origin, dest, fl_date, scheduled_elapsed_minutes)* was created. Placing the sort column last in the composite index allows the planner to return rows in sorted order from the index traversal itself. A secondary experiment confirmed that skipping the leading index columns (querying by fl_date alone without origin/dest) caused the planner to ignore the index and revert to a sequential scan, illustrating the leading-column constraint of composite B-tree indexes.
   - **Observation:** Index reduced sorting overhead and combined sorting and filtering to improve query performance. Execution time reduced from 1858.596 ms to 0.119 ms using Index Scan.
3. **Feature 3: Cheapest Flight Per Route/Day (Partial Index):**
   - **Application behavior:** For a given route over a date range, find the single cheapest available (non-cancelled) flight for each date.

     *SELECT DISTINCT ON (fl_date) id, fl_date, price, origin, dest FROM flights*
     *WHERE origin='SFO' AND dest='MIA' AND fl_date BETWEEN '2025-01-01' AND '2025-01-31'*
     *AND cancelled=false ORDER BY fl_date, price;*
   - **Mapping:** PostgreSQL uses heap storage; a partial index filters only the relevant subset of rows at index creation time. Index *ix_route_daterange_price ON flights (origin, dest, fl_date, price) WHERE cancelled = false* was created. This partial index includes only non-cancelled rows, making it smaller and faster to scan than a full index.
   - **Observation:** Since the vast majority of application queries filter on cancelled = false, the partial index provides index scan performance while consuming less storage and maintenance overhead than an equivalent full index.

4. **Feature 4: Covering Index and Index-Only Scan:**
    - **Application behavior:** Aggregate the minimum price and minimum duration per date for a given route over a date range

      *SELECT fl_date, MIN(price) AS lowest_price, MIN(scheduled_elapsed_minutes) AS fastest_minutes*
      *FROM flights WHERE origin='JFK' AND dest='LAX'*
      *AND fl_date BETWEEN '2025-01-01' AND '2025-01-31' GROUP BY fl_date ORDER BY fl_date;*
    - **Mapping:** For the Plain index (without INCLUDE), PostgreSQL performs an index scan followed by heap fetches to retrieve scheduled_elapsed_minutes, which is not in the index. Index-only scan is blocked because heap access is required. A Covering index ix_cover ON flights (origin, dest, fl_date, price) INCLUDE (scheduled_elapsed_minutes) was created. After VACUUM ANALYZE updates the visibility map, PostgreSQL performs a true index-only scan.
    - **Observation:** Index only scan revealed Heap Fetches: 0 in the EXPLAIN output. All required data is read from index leaf pages alone, eliminating heap I/O entirely and reducing disk access significantly on large datasets.
5. **Feature 5: Departure Time Range Filter:**
    - **Application behavior:** Find flights on a given route and date departing within a specified time window.

      *SELECT * FROM flights WHERE origin='ATL' AND dest='DEN' AND fl_date='2025-01-22'*
      *AND scheduled_dep_time BETWEEN '08:00:00' AND '12:00:00' ORDER BY scheduled_dep_time;*
    - **Mapping:** PostgreSQL uses heap storage; the time range filter requires a range scan on scheduled_dep_time after equality filters narrow the search. ix_dep_time ON flights (origin, dest, fl_date, scheduled_dep_time) was created. The time range filter maps to a B-tree range scan on the trailing index column after the leading equality conditions narrow the search to a specific route and date.
    - **Observation:** Without the index, the planner performs a full sequential scan + sort by departure time. With the index, it uses an index scan with ordered retrieval, returning results pre-sorted by departure time.
6. **Feature 6: Cancellation Rate by Airline (Aggregation Index):**
    - **Application behavior:** For a given route, compute each carrier's total flights, cancelled count, and cancellation rate percentage, helping users choose reliable airlines.

      *SELECT carrier, COUNT(*) AS total_flights,*
      *COUNT(*) FILTER (WHERE cancelled=true) AS total_cancelled,*
      *ROUND(100.0 * COUNT(*) FILTER (WHERE cancelled=true) / COUNT(*), 2) AS cancellation_rate*
      *FROM flights WHERE origin='SEA' AND dest='DFW' GROUP BY carrier ORDER BY cancellation_rate;*
    - **Mapping:** The index created was ix_can_rate ON flights (origin, dest, carrier, cancelled). Without Index, it performs a full sequential scan + Hash aggregate. With index, Index Scan on (origin, dest) narrows the route, then carrier and cancelled values are read from the index for aggregation. This feature illustrates

a case of low selectivity where cancelled is a boolean column with only two possible values, so the planner may choose a sequential scan over an index scan if it estimates the index overhead exceeds the benefit.

- **Observation:** This feature demonstrates that the query planner dynamically weighs index utility based on data distribution and selectivity. The cost model correctly prefers a sequential scan on small datasets and switches to an index scan on the full dataset.

---

## 6. Comparison with MySQL and MongoDB:

The following table shows the comparison of the three DB systems across various features:

| **Attributes** | **PostgreSQL** | **MySQL** | **MongoDB** |
|---|---|---|---|
| Storage Model | Heap files + secondary B-tree indexes | InnoDB: clustered B-tree (PK-ordered) | BSON (Binary JSON) documents, collection-level storage |
| Index Type | B-tree, Hash, GIN, BRIN, partial, covering | B-tree (InnoDB), FULLTEXT, SPATIAL | B-tree on _id and user-defined fields |
| Query Execution | Cost-based planner, EXPLAIN ANALYZE | Cost-based, limited plan introspection | Query optimizer, explain() for plans |
| ACID Support | Full ACID with MVCC | Full ACID with InnoDB MVCC | Multi-doc ACID since v4.0 (limited) |
| Schema | Strict relational schema | Strict relational schema | Flexible / schema-less documents |
| Joins | Full join support (hash, nested loop, merge) | Full join support | No native joins; uses $lookup which is slow |
| Scalability | Vertical; logical replication | Vertical; read replicas | Horizontal sharding built-in |
| Best For | Complex queries, OLTP, analytics | Web apps, simple OLTP workloads | Flexible document storage, rapid development |

The InnoDB storage engine of MySQL uses a clustered index where the records are stored physically by the value of their primary keys, hence making primary key range queries very efficient. But even with secondary indexes, a whole record needs to be retrieved using the primary key, just like a heap fetch. PostgreSQL does not have any clustered indexes but instead uses INCLUDE to achieve a similar purpose through covering indexes. MongoDB's document model is a poor fit for this application. Flight data is clearly structured and relational in nature, and the queries utilized by the program require multiple conditions and aggregation. Without supported join operations, the recreation of the same query in MongoDB requires denormalization and additional logic, which is much more complicated than one SQL query. For

features of this project such as high-selectivity multi-condition filters on structured tabular data with ORDER BY on a trailing index column, the composite and covering B-tree indexes of PostgreSQL prove superior to secondary indexes of MySQL and query model of MongoDB. In addition, PostgreSQL provides better query introspection using EXPLAIN ANALYZE.

---

## 7. Limitations & Lessons Learned:

1. **Limitations**:
    a. The dataset uses a synthetic price column generated from distance and duration rather than real fare data, not reflecting actual airline pricing complexity.
    b. For very small result sets (e.g., a single route on a single day with few matching rows), the performance difference between sequential scan and index scan is minimal and sometimes reversed, since index overhead can exceed the cost of scanning a small table.
    c. Index maintenance overhead (INSERT, UPDATE, DELETE performance degradation with multiple indexes) was not benchmarked, as the project focused on read-heavy search workloads
2. **Lessons Learned**:
    a. Column order in composite indexes matters. Putting filter columns (origin, dest, fl_date) before sort columns (price, elapsed_minutes) allows PostgreSQL to narrow the scan and skip an explicit sort in one traversal.
    b. Partial indexes work best on queries with frequent predicates. Indexing only cancelled = false rows shrinks index size and reduces maintenance overhead for the app's most common query pattern.
    c. Covering indexes (INCLUDE) eliminate heap access, but require VACUUM ANALYZE to update the visibility map before the planner will use an index-only scan. This operational dependency is always to be considered.
    d. The planner doesn't always pick a created index. For low-selectivity columns like a boolean cancelled, a sequential scan may genuinely be cheaper. Cost estimation matters as much as index design.
    e. EXPLAIN ANALYZE with BUFFERS is essential. Shared hit counts, planning time, and execution time give concrete before/after evidence of index impact which are far more reliable than theoretical estimates.

---

## 8. Project Links:

All project source code, SQL scripts, the data preprocessing pipeline, the Streamlit UI, and setup instructions README file having environment setup, dependency installation, database configuration, are available in the GitHub repository listed below.
**GitHub Repository**: https://github.com/Rylannat/DSCI_551_FINAL_PROJECT
**PostgreSQL Documentation:** https://www.postgresql.org/docs/current/indexes.html